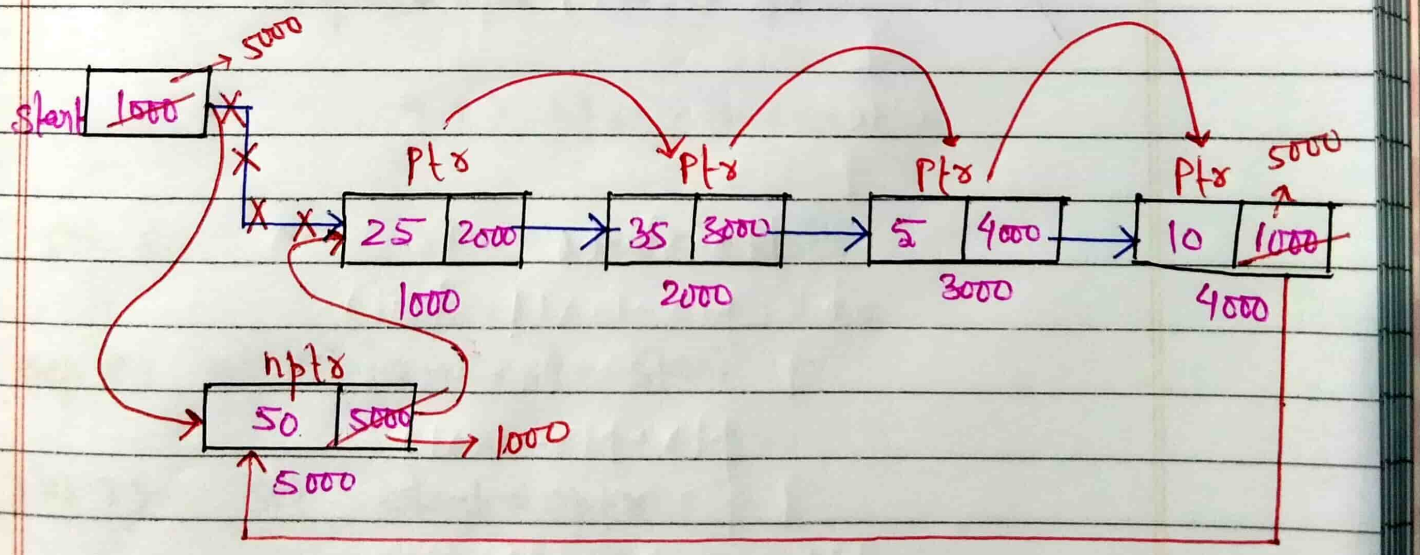
Insertion into a circularly singly linked list :

Insertion into a circularly singly linked list refers to the process of adding a new node to the list while maintaining the circular nature of the structure.

Types of insertion:

1.) Insertion at the begining :

A new node is inserted before the first node, and the last node's next pointer is updated to point to the new node.



Algorithm :

Step 1.) Create nptr [new-node]

~~Step 2.) Set ptr = start~~

Step 2.) IF start = NULL

2.1. set start = nptr

2.2.) set nptr→next = start

Step 3.) else,

set ptr = start

Step 4.) Repeat until ptr→next != start

4.1.) ptr = ptr→next

Step 5.) set ptr→next = nptr

Step 6.) set nptr→next = start

Step 7.) set start = nptr

Step 8.) Exit

Akash

→ pseudo code for insertion at begining in a circularly singly linked list:

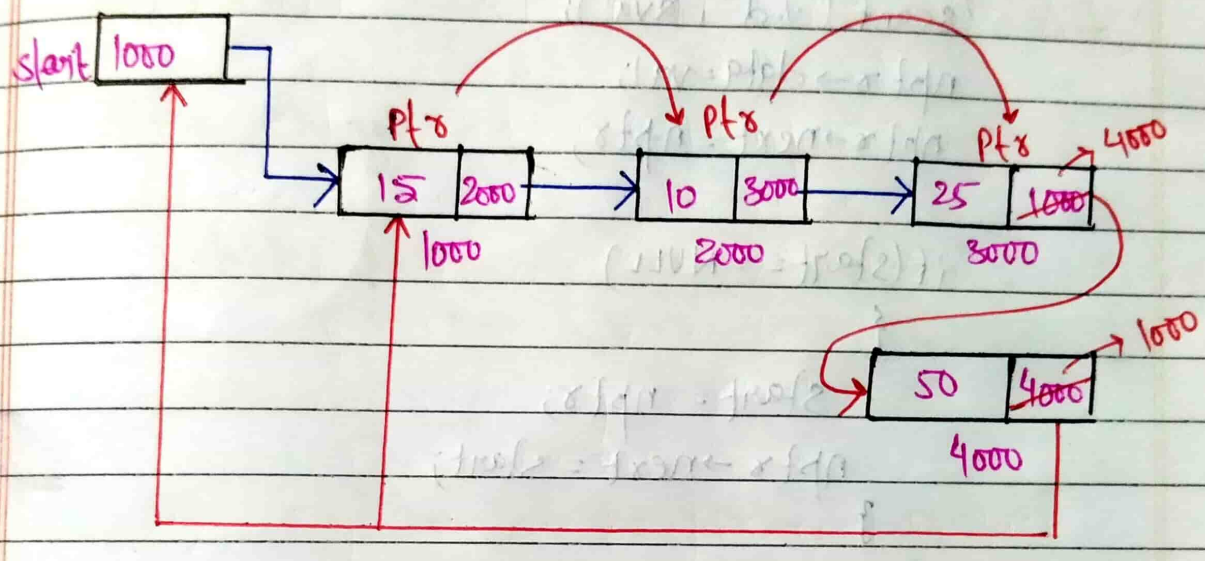
```
struct node *ptr, *nptr;
int val;
nptr = (struct node *) malloc (sizeof (struct node));
printf("enter value to insert);
scanf("%d", &val);
nptr->data = val;
nptr->next = nptr;

if (start == NULL)
  {
     start = nptr;
     ~~nptr = start;~~ nptr->next = start
  }
else
  {
     ptr = start;
   while (ptr->next != start)
     {
        ptr = ptr->next)
     }
   ptr->next = nptr;
     nptr->next = start ;
       start = nptr;
  }
   return start;
```

Akash

2.> Insertion at the end:

A new node is inserted after the last node, and the new node's next pointer is updated to point to the head.



Algorithm:

Step 1.> Create nptr [new_node]

Step 2.> IF start == NULL

2.1.> start = nptr

2.2.> nptr → next = start

Step 3.> else,

set ptr = start

~~Step 4.~~ 3.1.> ptr = ptr → next

Step 4.> set ptr → next = nptr

Step 5.> set nptr → next = start

Step 6.> Exit

Akash

→ Pseudo code for insertion at end :

```
struct node *ptr, *nptr;
int val;
nptr = (struct node *) malloc(sizeof(struct node));
printf("enter value to insert at end");
scanf("%d", &val);
nptr->data = val;
nptr->next = nptr;

if(start==NULL)
  {
    start = nptr;
    nptr->next = start;
  }

else
  {
    ptr = start;
    while(ptr->next != start)
      {
        ptr = ptr->next;
      }
    ptr->next = nptr;
    nptr->next = start;
  }
return start;
```

Akash

→ Insertion at the specific value:

A new node is inserted ~~at a given position~~ ~~is the~~ after a given value in the list.

Same as singly linked list;

Little change;

```
while( ptr->next != start && ptr->data != num)
        {
            ----
        }
if ( ptr == start)
    {
        ---
    }
```

Akash

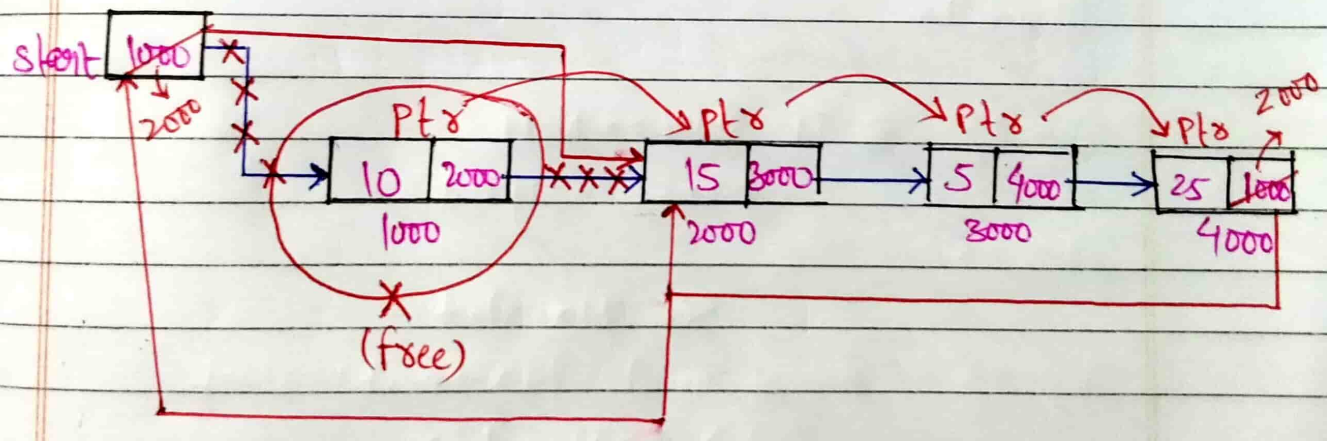
Deletion into a circular singly linked list:

Deletion in a circularly singly linked list means removing a node from list while maintaining its circular structure.

There are three common cases of deletion:

1.) Deletion at the begining:

The first node is remove and the last node is updated to point to the new first node (next node).



Algorithm:

Step 1.) IF start == NULL
print "empty list

Step 2.) else,
set ptr = start

Step 3.) repeat until ptr → next != start
3.1. set ptr = ptr → next

Akash

Step 4.> set ptr → next = start → next

Step 5.> free (start)

Step 6.> set start = ptr → next

Step 7.> Exit.

Akash

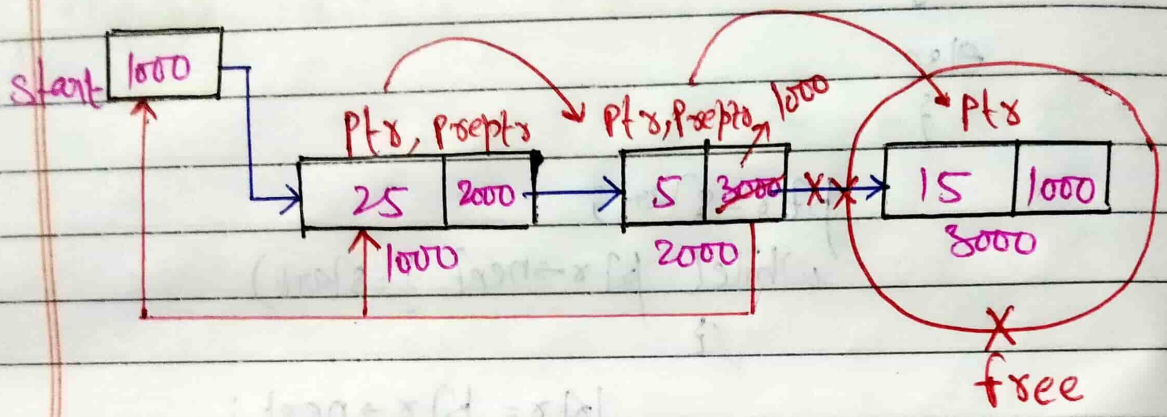
→ Pseudo code for deletion at begining

```
struct node *ptr;
if (start==NULL)
  {
      printf("empty list");
  }
else
  {
      ptr=start;
      while( ptr->next !=start)
        {
            ptr= ptr->next;
        }
      ptr->next = start->next;
        free(start);
        start = ptr->next;
  }
return start;
```

Akash

2.) (D)eletion at end :

The last node is removed, and the second-last node is updated to the point to the start.



Algorithm:

Step 1.) IF Start == NULL
print "empty list"

Step 2.) else,
set ptr = start

Step 3.) Repeat until ptr → next != start
3.1.) preptr = ptr
3.2.) ptr = ptr → next

Step 4.) set Preptr → next = start
Step 5.) free (ptr)
Step 6.) Exit

Akash

→ Pseudo code deletion at end :

```
struct node *preptr, *ptr;
if(start == NULL)
  {
    printf("empty list");
  }
else
  {
    ptr=start;
    while(ptr->next != start)   // NULL ⇌ start
      {
        preptr = ptr;
        ptr=ptr->next;
      }
    preptr->next = start;   // start ⇌ NULL
    free(ptr);
  }
return start;
```

// Same as single linked list

→ Pseudo code for deletion at specific value;

Same as singly linked list

Chang→ while(ptr != start && ptr->data != val)
{ ___
if (ptr == start)
{ ___